



Universidade Federal de Mato Grosso do Sul
Faculdade de Computação

Disciplina: Arquitetura de Computadores II
Professor: Renan Albuquerque Marks

Descrição do Trabalho Prático

1 Introdução

Neste documento estão detalhados os procedimentos que devem ser seguidos para o desenvolvimento do Trabalho da disciplina de Arquitetura de Computadores II. Este trabalho será desenvolvido ao longo deste semestre letivo e constituirá como parte da nota final da disciplina de Arquitetura de Computadores II.

É fortemente recomendado que os estudantes acessem com frequência este documento para esclarecer possíveis dúvidas, estar ciente do cronograma e estar a par de possíveis atualizações/alterações no trabalho.

2 Objetivo

O objetivo deste trabalho prático é mostrar na prática como a análise de complexidade de algoritmos está diretamente relacionada com a quantidade de instruções executadas no hardware após a implementação e compilação utilizando diversos níveis de otimização. Além disso, consegue exemplificar como o tempo de acesso à memória impacta diretamente no tempo de execução, principalmente em situações nas quais a quantidade e qualidade das falhas nos acessos às caches pode degradar o desempenho mesmo quando a implementação é feita em linguagens compiladas.

3 O que deve ser feito?

Neste trabalho, deve-se fazer a análise de desempenho de algoritmos de ordenação em diferentes implementações usando diferentes linguagens de programação. As linguagens que deverão ser usadas são **Python 3**, **C/C++** e **Java**.

Sua análise de desempenho deverá ser feita dentro do sistema operacional Linux usando o script **benchmark.sh** presente no mesmo pacote ZIP dessa descrição. A distribuição Linux a ser usada é de livre escolha, desde que possua instalado os softwares **perf**, Python 3 com o pacote **matplotlib**.

Cada grupo deve escolher **dois** algoritmos de complexidades teóricas **diferentes** para realizar a implementação em **três** linguagens **diferentes**. Isto é: cada algoritmo deve ser implementado três vezes, uma vez em cada linguagem. Por exemplo: O algoritmo Bubble Sort deve ser implementado em C/C++, Python e Java e ter o desempenho analisado nestas três versões. Os algoritmos disponíveis estão listados na Tabela 1.

Complexidade Caso-Médio	Algoritmo (Link)
$O(n \log n)$	Quick Sort
$O(n \log n)$	Merge Sort
$O(n \log n)$	Heap Sort
Depende	Shell Sort
$O(n^2)$	Gnome Sort
$O(n^2)$	Insertion Sort
$O(n^2)$	Cocktail Sort
$O(n^2)$	Selection Sort
$\Omega(n^2/2^p)$	Comb Sort
$O(d \cdot n)$	Radix Sort

Tabela 1: Listagem de algoritmos de ordenação.

3.1 Critérios para implementação

Os algoritmos **devem** ser implementados **seguindo** os seguintes critérios a fim de ser feita uma comparação justa de desempenho entre as diferentes linguagens usadas:

1. Os algoritmos **devem** ordenar um vetor unidimensional de 6800 posições;
2. O vetor **deve** ser alocado como variável **global**, isto é, **fora** de funções;
 - No caso do Java, o vetor deve ser alocado como membro/escopo da classe.
3. O vetor **não deve** ser alocado como variável **dinâmica**, isto é, não deve ser usado `malloc()` ou `new()` (somente C/C++);
4. O vetor **não deve** ser alocado com tipo `std::vector`. No lugar, deve-se usar o tipo `std::array` (somente C++);
5. O vetor **deve** ter todas suas posições preenchidas em **tempo de compilação**;
6. O vetor **não deve** ser preenchido durante o **tempo de execução** (isto é, logo antes da ordenação ser chamada);
7. Todas as implementações **devem** utilizar a **mesma configuração** de vetor, isto é, os **valores aleatórios** e suas **respectivas posições** no vetor **antes** da invocação da ordenação **devem ser as mesmas**;

Um exemplo de implementação em C que segue todos os critérios apresentados pode ser visto na Figura 1.

4 Execução do script e coleta de dados

O script `benchmark.sh` está implementado de forma a executar os arquivo executaveis da sua implementação 10 vezes através do software `perf` para a coleta dos índices de desempenho. Ao final da execução, o script usará as informações coletadas para gerar

```

1  int a[6800] = {1197, 5121, 5741, ... , 5287, 6538, 7432};
2
3  void bubbleSort(int* vetor, int tamanho) {
4      for (int i = 0; i < tamanho - 1; i++) {
5          for (int j = 0; j < tamanho - i - 1; j++) {
6              if (vetor[j] > vetor[j + 1]) {
7                  int temp = vetor[j];
8                  vetor[j] = vetor[j + 1];
9                  vetor[j + 1] = temp;
10             }
11         }
12     }
13 }
14
15 int main(){
16     bubbleSort(a, MAX);
17     return 0;
18 }

```

Figura 1: Exemplo de implementação em linguagem C do Bubble Sort.

gráficos informativos de forma a se proceder com a análise do desempenho dos programas. Ao executar o script com a opção **benchmark.sh --help** ele mostrará as instruções de uso vistas na Figura 2.

Caso a implementação seja em C/C++, o script executará o(s) arquivo(s) executável(eis) (permissão **+x**) que esteja(m) dentro do diretório de destino. Por exemplo, caso a estrutura de pastas seja a mostrada na Figura 3, ao se executar o script com o comando **benchmark.sh --all Bubble**, o script executará o **perf** para cada um dos executáveis de nome “bubblesort a estrutura de diretórios se tornará a mostrada na Figura 4.

O arquivo **Grafico_Bubble_20260325_181254.pdf** contem os gráficos gerados a partir das saídas das execuções obtidas pelo **perf** e que foram salvas no arquivo **resultado_Bubble_20260325_181132.txt**. O conjunto de gráficos gerados pode ser visto na Figura 5.

Importante!

Caso sua CPU seja um modelo que possua diferença arquitetural entre os núcleos como, por exemplo, núcleos de desempenho e núcleos de eficiência (como nos modelos mais recentes de processadores da Intel, AMD e ARM), é recomendado executar o script fixando a execução para que use somente um núcleo específico. Caso isso não seja feito, o sistema operacional irá escalonar a execução em diversos núcleos e isso pode gerar medições de desempenho incorretas pelo aplicativo **perf**.

```

./benchmark.sh --help
Uso: ./benchmark.sh [OPÇÕES] [DIRETÓRIOS...]
Opções:
  --cache      Mede eventos de L1 e LLC Cache
  --branches   Mede predições e falhas de Branch
  --all        Mede todos os eventos disponíveis
  --plot-only  Apenas gera gráficos a partir de arquivos .txt existentes na pasta

Exemplos:
  Rodar testes: ./benchmark.sh --all Bubble
  Apenas plotar: ./benchmark.sh --plot-only resultados_20260226_125343
  Caso seu processador tenha núcleos híbridos, experimente usar taskset.
  Exemplo: taskset --cpu-list 1 ./benchmark.sh --all Bubble

```

Figura 2: Mensagem de instruções de uso do script de medição de desempenho.

```

Trabalho/
├── Bubble/
│   ├── bubblesort_o0
│   ├── bubblesort_o1
│   ├── bubblesort_o2
│   ├── bubblesort_o3
│   ├── bubblesort_os
│   └── bubblesort_of
└── benchmark.sh

```

Figura 3: Listagem de diretórios de exemplo **antes** da execução do script `benchmark.sh`.

```

Trabalho/
├── Bubble/
│   ├── bubblesort_o0
│   ├── bubblesort_o1
│   ├── bubblesort_o2
│   ├── bubblesort_o3
│   ├── bubblesort_os
│   └── bubblesort_of
├── resultados_Bubble_20260325_181132/
│   ├── Grafico_Bubble_20260325_181254.pdf
│   └── resultado_Bubble_20260325_181132.txt
└── benchmark.sh

```

Figura 4: Listagem de diretórios de exemplo **depois** da execução do script `benchmark.sh`.

Bubble (Média e SD - 100 Runs)

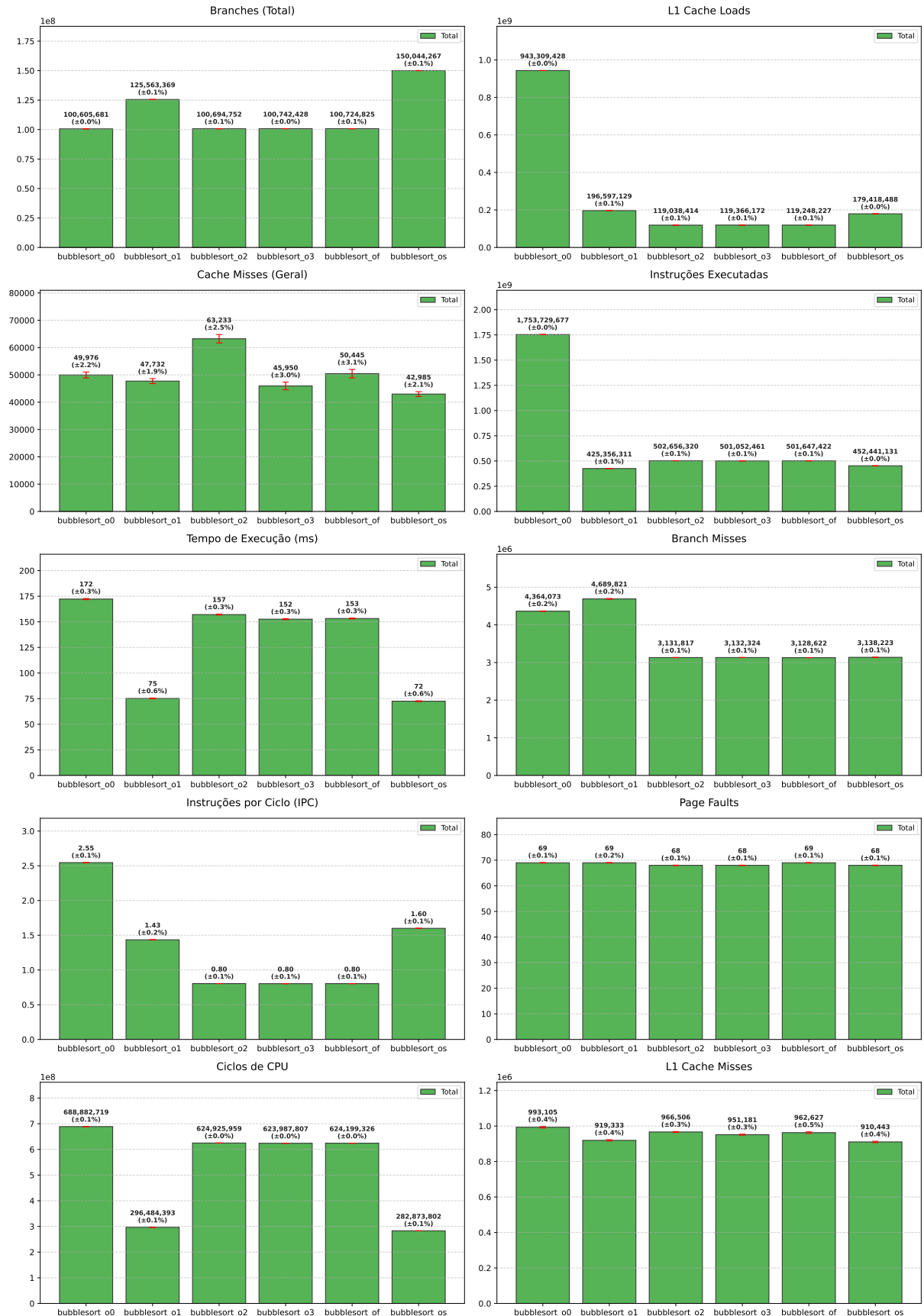


Figura 5: Conjunto de gráficos gerados pelo script.

Essa execução fixada em somente um núcleo pode ser conseguida utilizando o utilitário **taskset**. Por exemplo, para fixar a execução do processo no núcleo 1 da CPU, basta executar:

```
taskset --cpu-list 1 ./benchmark.sh --all Bubble
```

A medição de desempenho em cada linguagem deve seguir às seguintes regras:

4.1 Implementações em C/C++

Os algoritmos escolhidos devem ser compilados em 6 (seis) executáveis, cada um utilizando uma das 6 (seis) otimizações disponíveis no compilador gcc/g++:

- O0** Otimizações desligadas;
- O1** Otimizações básicas ligadas;
- O2** Otimizações básicas e avançadas ligadas;
- O3** Otimizações básicas, avançadas e vetoriais ligadas;
- Os** Otimizações para redução de tamanho de código;
- Of** Otimizações para execução rápida (sem redução de tamanho);

4.2 Implementações em Python

Utilizar o recurso de *shebang* do shell para produzir um script executável diretamente. Para isso, a primeira linha deve ser um comentário apontando para o caminho do interpretador a ser usado na execução do script. Para scripts em Python, um exemplo de *shebang* pode ser visto na Figura 6.

```
1 #!/bin/python3  
2  
3 print('Esse script será executado diretamente pelo Python.')
```

Figura 6: Exemplo de cabeçalho *shebang* em script Python.

Como a linguagem Python não possui opções de compilação e otimizações, a medição de desempenho será feita diretamente com a execução do script.

4.3 Implementações em Java

No caso do Java, não é possível utilizar o recurso de *shebang*, visto que programas escritos em Java são compilados, tais quais os programas em C/C++. Porém há uma diferença: em java é necessário invocar a JVM passando como parâmetro o nome da classe principal compilada a ser executada. Para isso, basta criar um script bash executavel na mesma

```
1 #!/bin/bash
2
3 java Main
```

Figura 7: Exemplo de script bash invocando JVM para execução da classe Main compilada no arquivo **Main.class**.

pasta que se encontra o arquivo **.class** e invocar a JVM por ele. Um exemplo de script *shebang* que invoca a JVM pode ser visto na Figura 7.

Como a linguagem Java não possui opções de otimizações na compilação (pois as otimizações são feitas em tempo de execução), a medição de desempenho será feita diretamente com a execução do script.

5 Artefato produzido

Ao final da coleta dos dados e produção dos gráficos, deverá ser produzido um relatório em $\text{\LaTeX}$ usando o template da Sociedade Brasileira de Computação e entregue em formato PDF. Este relatório deve possuir até o **limite máximo de 10 páginas**.

O relatório deve conter as seguintes seções com os respectivos conteúdos:

1. Introdução;

- Algoritmos analisados:
 - Nome;
 - Complexidade;
 - Breve descrição de funcionamento (1 parágrafo de até 5 linhas para cada algoritmo);
- Sistema Operacional utilizado: nome e versão;
- Hardware utilizado:
 - CPU (modelo e velocidade);
 - Quantidade de memória RAM (modelo e velocidade);
 - Armazenamento (modelo e capacidade);
- Compiladores e interpretadores: versões utilizadas;

2. Resultados

- Uma seção para cada algoritmo contendo:
 - Os gráficos produzidos;
 - Análise dos resultados (exemplo pode ser visto na seção 6 [deste trabalho](#));

3. Conclusão

- Fatos encontrados baseados nos dados obtidos.

4. Dificuldades encontradas

- Até 10 linhas descrevendo as dificuldades encontradas no desenvolvimento do trabalho e as soluções encontradas e adotadas para resolvê-las.

6 Atribuição de grupos

Os trabalhos deverão ser feitos por grupos contendo no máximo 2 (dois) participantes. A cada grupo será sorteado dois algoritmos com complexidades diferentes para implementação. A atribuição dos algoritmos para cada grupo será postada no AVA da disciplina.

7 Cronograma

- **Início dos trabalhos:** 02/04/2026;
- **Entrega:** 10/05/2026 até as 23h59 — Cada grupo deve submeter, via AVA:
 - Relatório em PDF;
 - Código-fonte das implementações;
 - Executáveis compilados (caso aplicável);

8 Avaliação do trabalho

- **Nota da Metodologia:** valor no intervalo $[0,5]$ que será atribuída de acordo com a aderência aos critérios de implementação listados nas Seções 3.1 e 4.
- **Nota do Relatório:** valor no intervalo $[0,5]$ que será atribuído ao conteúdo do relatório listado na Seção 5.
- A nota final do trabalho prático será composta da soma das notas da metodologia e relatório;
- Trabalhos que **não** compilarem e/ou executarem receberão nota **zero**;
- **Atenção:** Não serão aceitas entregas de trabalho atrasadas.
- **Casos de Plágio nos códigos serão tratados com rigor.**
- **Uso de LLMs para escrita do trabalho não é permitido.**

9 Agradecimentos

Gostaria de deixar registrado agradecimentos especiais ao aluno **Pedro Paulo de Oliveira Andrade** do curso de **Ciência da Computação** que foi o responsável pelo desenvolvimento, teste e validação do script **benchmark.sh** como resultado da Atividade Orientada de Ensino desenvolvida por ele durante os semestres 2025/2 e 2026/1. Obrigado Pedro!

10 Dicas e Sugestões

- Inicie o trabalho o **quanto antes**. O tempo **voa**!
- Retire as dúvidas quanto ao entendimento dos elementos que compõem a metodologia. Isso possibilitará detectar possíveis falhas na implementação logo cedo.
- Trabalhem em equipe!
- Para a escrita do trabalho, pode-se ser utilizado tanto **Overleaf** quanto o **Latex UFMS** (Overleaf local sem limitações).
- É permitido o uso de LLM gerativas para estudo, discussão e interpretação dos resultados, mas não para escrita do relatório.